SOFTWARE ENGINEERING PROJECT

Prestimulus EEG-Based Behavioral Prediction in the HBN-EEG Dataset

BY

Nantawat Suksirisunt
Naytitorn Chaovirachot

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY

Academic Year 2565

Prestimulus EEG-Based Behavioral Prediction in the HBN-EEG Dataset

BY

Nantawat Suksirisunt
Naytitorn Chaovirachot

This Project Submitted in Partial Fulfillment of the Requirement for Bachelor Degree of Engineering (Software Engineering)
Department of Computer Engineering, Faculty of Engineering KASERTSART UNIVERSITY
Academic Year 2565

Approved By:

Advisor .... **Date** ....
(Asst. Prof.Dr. Thanawin Rakthanmanon)

Co-Advisor .... **Date** ....
(Assoc. Prof. Dr. Theerawit Wilaiprasitporn)

Head of Department .... **Date** ....
(Assoc. Prof.Dr. Punpiti Piamsa-Nga)

Abstract

Electroencephalography (EEG) is widely used to study how neural activity relates to behavior, but practical EEG research often depends on ad-hoc exploratory workflows. Interactive notebooks are convenient for inspection and rapid iteration; however, hidden execution state, environment dependency, and inconsistent preprocessing make it difficult to reproduce results, compare experiments systematically, and share a workflow that others can run in the same way.

This project investigates whether prestimulus EEG can predict behavioral performance in the Healthy Brain Network EEG (HBN-EEG) dataset. We focus on the Contrast Change Detection (CCD) task and use the 2-second inter-trial interval (ITI) segment as a constrained prestimulus window for modeling.

The proposed approach extracts steady-state and spectral features from CCD-ITI epochs after standard preprocessing (e.g., bandpass filtering, artifact mitigation, and normalization). We train models to predict reaction time (RT) as a regression target and trial-level correctness (accuracy) as a binary classification target. Performance is evaluated using regression metrics for RT (e.g., MAE/MSE) and classification metrics for correctness (accuracy, F1-score, precision, and recall).

To support reproducible experimentation, the software component provides a lightweight interface for configuring preprocessing, running feature extraction and modeling, and producing standardized outputs (tables/plots) without relying on notebook-based trial-and-error.

Keywords: Prestimulus EEG, HBN-EEG, Contrast Change Detection, Reaction Time, Correctness Prediction, Inter-trial Interval, EEG Preprocessing

Acknowledgement

Put your acknowledgement paragraph here.

Nantawat Suksirisunt
Naytitorn Chaovirachot

Table of Content

Table of Table

Table of Figure

Chapter 1

Introduction

This chapter introduces the motivation for prestimulus EEG-based behavioral prediction and explains why the HBN-EEG CCD task is an appropriate case study. In addition to the research question, it motivates the software need: a reproducible, end-to-end workflow that reduces reliance on fragmented notebooks and makes experiments easier to rerun, compare, and report.

1.1 Background

Electroencephalography (EEG) is widely used in neuroscience and brain-computer interface (BCI) research to study brain activity with high temporal resolution. Modern EEG datasets, such as those following the Brain Imaging Data Structure (BIDS), often contain large volumes of multi-subject recordings with complex experimental conditions. Analyzing such data typically involves multiple stages, including signal inspection, preprocessing, feature extraction, and model development.

In current research practice, these workflows are commonly implemented using notebook-based environments. While flexible, this approach requires manual scripting for each step, leading to fragmented processes and limited reproducibility. Researchers must repeatedly configure preprocessing pipelines, adjust parameters, and re-run computations, which becomes increasingly inefficient as dataset size and complexity grow.

Furthermore, tasks such as comparing signals across subjects or exploring cohort-level patterns require additional manual effort. As a result, the overall workflow becomes time-consuming and computationally expensive, often requiring significant hardware resources for repeated execution.

This work utilizes the Healthy Brain Network EEG (HBN-EEG) dataset, which provides high-density 128-channel recordings across multiple experimental tasks. The dataset is distributed in BIDS format and incorporates Hierarchical Event Descriptors (HED), supporting standardized and reproducible EEG analysis. It is provided through the EEG Foundation Challenge (EEG2025), highlighting the growing need for scalable and efficient analysis tools. [1]

1.2 Problem Statement

Despite the availability of powerful EEG processing libraries, the current workflow for EEG data analysis remains inefficient and difficult to scale. The reliance on notebook-based pipelines introduces several key challenges.

First, setting up and executing preprocessing and analysis pipelines requires substantial time and effort. Even minor changes in parameters often necessitate re-running the entire computation, resulting in slow iteration cycles.

Second, existing workflows provide limited support for systematic comparison across subjects. Researchers must manually select and process individual datasets, making cohort-level analysis cumbersome and error-prone.

Third, the computational cost associated with repeated preprocessing and model experimentation can be high. This not only slows down research progress but also may require significant computational resources.

These limitations hinder efficient exploration of EEG data and slow down the development of machine learning models for tasks such as behavioral prediction

[1] See [1, 2].

1.3 Solution Overview

To address these challenges, this thesis proposes an interactive EEG analysis platform designed to streamline the research workflow from data inspection to model experimentation.

The platform integrates data selection, preprocessing, visualization, and machine learning experimentation into a unified, parameter-driven workflow. By centralizing these functionalities, the system enables users to perform complex analysis tasks without relying on manual scripting.

A key design feature of the platform is the mode-action interface, which organizes functionality into four primary operational modes: Plot Mode, Grid Plot Mode, Data Mode, and AI Mode. Each mode provides context-specific actions, allowing users to focus on relevant tasks while maintaining a structured workflow.

The platform also supports flexible subject selection and cohort filtering. Users can perform detailed analysis on individual subjects or apply attribute-based filters to explore patterns across multiple participants. This capability facilitates both fine-grained inspection and large-scale exploratory analysis.

In addition, the platform provides integrated support for machine learning experimentation, including model training and basic evaluation. This allows users to iteratively refine models within the same environment used for data exploration.

Overall, the system is designed to enable more efficient iteration during EEG research by reducing reliance on repetitive notebook-based processes.

1.4 Target User

The platform is designed for researchers and students working in fields related to neuroscience, brain-computer interfaces (BCI), and EEG data analysis. These users typically require tools for inspecting neural signals, preparing datasets, and developing machine learning models for experimental tasks.

The system is particularly beneficial for individuals who work with multi-subject EEG datasets and require efficient methods for comparing signals and conducting exploratory analysis.

1.5 Benefit

The proposed platform offers several key benefits.

First, it helps reduce the time required for EEG data analysis by minimizing repetitive manual scripting. Users can perform data selection, preprocessing, and visualization through an integrated interface, supporting faster iteration.

Second, the platform improves usability by providing a structured workflow through its mode-action design. This reduces complexity and helps users focus on specific analysis tasks.

Third, the system enhances reproducibility by maintaining a parameter-driven workflow and logging user actions. This ensures that analysis steps can be consistently replicated.

Finally, the integration of machine learning experimentation within the same environment allows users to seamlessly transition from data exploration to model development. This unified approach supports more efficient research workflows, particularly for tasks involving behavioral prediction from EEG signals.

1.6 Scope of the Study

This study focuses on the design and development of an interactive EEG analysis platform that supports data exploration, preprocessing, visualization, and machine learning experimentation within a unified workflow. The system is intended to facilitate efficient analysis of multi-subject EEG datasets, particularly those structured in standardized formats such as BIDS.

The scope of the platform includes functionality for single-subject inspection and cohort-level analysis through flexible filtering mechanisms. It also supports the configuration and execution of preprocessing pipelines, as well as model training and basic evaluation using metrics such as training loss, validation loss, mean absolute error (MAE), and coefficient of determination (R^2).

The platform is demonstrated using the Healthy Brain Network EEG (HBN-EEG) dataset, with a focus on behavioral prediction tasks, including contrast change detection (CCD). This serves as a representative use case to validate the system's capability in supporting real-world EEG research workflows.

However, this study does not aim to develop or optimize state-of-the-art machine learning models. The performance of predictive models is not the primary focus, and the platform is designed to be model-agnostic. Additionally, the system does not address real-time EEG processing, clinical validation, or deployment in medical environments.

1.7 Terminology

- **SSVEP:** Steady-State Visual Evoked Potential, frequency-locked EEG response to periodic stimulation.
- **ITI:** Inter-Trial Interval, rest period between trials; in CCD, flickering gratings persist during ITI.
- **BIDS/HED:** Data and event annotation standards supporting consistent, analysis-ready datasets.

1.8 Personal Development Plan

This project is used as a competency-building exercise in software and AI engineering. The personal development goals are to:

- strengthen the ability to design reproducible ML experimentation pipelines,
- improve EEG-domain engineering skills (event handling, epoching, spectral verification),
- practice writing maintainable software (layering, modular design, configuration management), and
- improve reporting skills by connecting method choices, software artifacts, and evaluation results in a single narrative.

Chapter 2

Literature Review and Related Work

This chapter reviews related work relevant to prestimulus EEG-based behavioral prediction and to the software workflow required to run such experiments reproducibly. It first summarizes competing tools/workflows and then reviews foundational resources and key methodological insights that informed this project.

2.1 Competitor Analysis

Several existing tools support EEG preprocessing and modeling, but they differ in how directly they support a constrained prestimulus prediction setting (CCD-ITI) and how well they support end-to-end reproducible experimentation.

- **MNE-Python / MNE-BIDS:** strong for EEG preprocessing and BIDS interoperability, but experiment tracking and UI-driven workflows are typically built by the user.
- **EEGLAB / Brainstorm:** mature GUI tools for analysis and visualization, but reproducible pipeline execution and code-centric experiment comparisons may require additional scripting.
- **General ML notebooks:** flexible and fast to prototype, but prone to fragmentation (hidden state, inconsistent preprocessing, unclear configurations) unless carefully standardized.

This project complements these tools by emphasizing a traceable workflow tailored to the CCD-ITI prestimulus window: consistent event parsing and trial alignment, standardized preprocessing and feature extraction, and repeatable evaluation for both RT regression and correctness classification.

2.2 Literature Review

2.2.1 Foundational Works Leveraged

The EEG Foundation Challenge (EEG2025) provides the Healthy Brain Network EEG (HBN-EEG) dataset as a large-scale, high-density, multi-task EEG resource. Importantly for reproducible research, the dataset is distributed in BIDS format with HED annotations, which standardize file organization and event labeling across recordings.[1]

- **HBN-EEG FAIR implementation (2024):** Presents analysis-ready EEG with integrated behavioral events and HED annotations, enabling reproducible analyses.[2]
- **HBN transdiagnostic resource (2017):** Describes the HBN biobank's large-scale and community-sampled design to support dimensional research.[3]
- **EEG + eye tracking paradigms (2017):** Provides tasks including steady-state and contrast decision tasks supporting developmental brain investigations.[4]

2.2.2 Key Insights and Adaptations from Recent Methodologies

Based on these resources, this project adopts the following practical principles:

- **Event-locked trial construction is critical:** prestimulus prediction requires careful alignment between events, EEG windows, and behavioral labels.
- **Verification before prediction:** steady-state characteristics should be validated (e.g., via SNR spectra) to ensure that extracted features correspond to measurable signal properties.

[1] [1, 2].
[2] [2].
[3] [3].
[4] [4].

- **Target-specific evaluation:** regression and classification require different metrics and error analyses; reporting should be consistent across runs.
- **Reproducibility as a software requirement:** pipeline parameters, dataset filters, and split strategies should be explicit to avoid misleading comparisons.

2.3 Positioning of This Work

This project uses HBN-EEG to study a focused research question: whether prestimulus EEG contains predictive information about behavioral performance. We model two outcomes: reaction time (RT) as a regression target and trial correctness (accuracy) as a binary classification target.

We evaluate this question in the Contrast Change Detection (CCD) task, where periodic visual stimulation during the rest period between trials supports steady-state feature extraction. By restricting inputs to a fixed 2-second prestimulus window, the modeling setting is intentionally constrained and emphasizes signal validation (via SNR spectra) and reproducible evaluation.

Chapter 3

Requirement Analysis

This chapter defines the requirements for the *EEG Workbench* software used in this project. The system is implemented as a notebook-first, Python-based tool (`eegkit`) that orchestrates loading HBN-EEG (BIDS-like releases), preprocessing, CCD trial/ITI epoching, dataset building, model training, and export of plots/tables/models. The focus is not only to "run experiments", but to make every run reproducible and auditable through cached pipeline artifacts and on-disk job specifications.

3.1 Stakeholder Analysis

The primary stakeholders of the proposed EEG analysis platform are researchers and students working in fields related to neuroscience, brain-computer interfaces (BCI), and EEG data analysis.

Researchers use the platform to conduct EEG data analysis, including signal inspection, preprocessing, and machine learning experimentation. Their work often involves multi-subject datasets and requires efficient tools for both individual and cohort-level analysis. Reproducibility and transparency are also critical, as experimental results must be consistent and verifiable.

Students use the platform for learning, experimentation, and project development. They often face challenges when working with complex EEG processing pipelines and scripting-based environments. The platform supports their needs by providing an interactive and structured interface that reduces technical barriers while still enabling advanced analysis capabilities.

In addition to primary stakeholders, the system also serves secondary stakeholders, such as machine learning practitioners or developers working with EEG data. These users may utilize the platform to prototype models, test data processing pipelines, or integrate new algorithms into the workflow. While they are not the main target users, the platform provides sufficient flexibility to support their experimentation needs.

Overall, the system is designed to support both experienced users and learners by simplifying EEG data workflows while maintaining the flexibility required for research and development.

3.2 Users and Interaction Modes

The workbench supports two interaction modes that correspond directly to the UI input schemas:

- **Single subject mode:** the user selects a subject, task name, and (optional) run identifier, then runs actions such as signal inspection plots, epoch extraction, and dataset/model generation.
- **Cohort filter mode (group):** the user selects a task and filters subjects by metadata (e.g., sex, age ranges) and task-derived CCD performance metrics (accuracy/response time). The user can optionally execute an action per subject to obtain comparable outputs.

3.3 User Stories

The following user stories describe the expected interactions between users and the system. These scenarios capture key functionalities required to support EEG data analysis and machine learning workflows.

- **US1 (Configure and load):** As a user, I want to point the tool to the HBN-EEG data directory and discover available subjects/tasks so that I can start analysis without manual path edits.

- **US2 (Inspect quality):** As a researcher, I want to preview raw/filtered EEG and view event tables so that I can verify signal quality and event integrity.
- **US3 (Reproducible preprocessing):** As a researcher, I want preprocessing/epoch parameters stored explicitly (band, notch, resample rate, channels, t_{min}/t_{max}, cleaning thresholds) so that results are comparable across runs.
- **US4 (Correct label alignment):** As a researcher, I want CCD trial outcomes to be aligned deterministically with trial anchors so that labels are not silently shifted.
- **US5 (Build datasets):** As a researcher, I want to build and save ML feature datasets and DL tensor datasets so that training can be repeated without re-deriving epochs each time.
- **US6 (Train/test and export):** As a researcher, I want model training/testing to output metrics and saved model artifacts so that I can compare approaches and report evidence.
- **US7 (Background execution):** As a user, I want long-running actions to run as background jobs with logs and saved outputs so that the notebook remains responsive.

3.4 Use Case Overview

The primary actor is the *Researcher*. The main use cases are: (1) configure and discover data, (2) inspect signals/events, (3) construct epochs and datasets, (4) train/test models, and (5) export and reproduce artifacts.

3.5 Use Case Model

3.5.1 Use Case 0: Configure Data Root

Goal: Configure the workbench to locate the HBN-EEG dataset on disk.

Main flow:

1. Set an environment variable (e.g., `DATA_DIR` / `EEG_DATA_DIR`) pointing to the dataset root.
2. Initialize the subject index, which discovers release folders and subject directories.
3. List available subjects/tasks to confirm correct data discovery.

3.5.2 Use Case 1: Inspect and Verify a Task Recording

Goal: Inspect a single subject/task/run and validate that events and signals look plausible.
Main flow:

1. Select *Single subject* input (subject, task, run).
2. Apply filtering/cleaning parameters (e.g., band-pass, notch, resample, channel selection).
3. View diagnostic plots (time domain, PSD/evoked/topomaps) and tables (events/channels/metadata).
4. Confirm that event markers needed for CCD preprocessing exist and are ordered correctly.

3.5.3 Use Case 2: Build CCD Epochs and Labels (Prestimulus/ITI)

Goal: Construct CCD trial-aligned epochs with deterministic outcome labels.
Main flow:

1. Select an epoch window (t_{min}, t_{max}) describing the prestimulus/ITI segment.
2. Create trial anchors from raw annotations (e.g., `contrastTrial_start`).

3. Derive trial outcome labels (hit/wrong/miss) from the parsed events table and strictly validate alignment to trial anchors.
4. Drop out-of-bounds epochs and interpolate bad channels when needed.
5. Cache derived epochs and labels so repeated analysis uses consistent data.

3.5.4 Use Case 3: Build Training Datasets

Goal: Build datasets that can be reused by training runs.
Main flow:

1. Choose dataset type:
   - **ML feature dataset:** extract per-epoch feature vectors from selected channels.
   - **DL epoch tensor dataset:** export epoch tensors shaped for neural networks.
2. Persist arrays `x`, `y`, and `group` (subject identifier) to the job output directory.
3. Save a dataset summary (shapes and class/group counts) for reporting and debugging.

3.5.5 Use Case 4: Train/Test Models and Export Artifacts

Goal: Train/test models and export both metrics and model artifacts.
Main flow:

1. Select a previously saved dataset (discovered under the `jobs/` directory).
2. Choose model type and hyperparameters.
3. Split data using group-aware strategies when possible (to avoid leakage across subjects).

4. Train and evaluate, then save: (i) model artifact, (ii) evaluation metrics JSON, and (iii) optional plots.

3.6 User Interface Design

The platform adopts a research-oriented interface design that aligns with the typical EEG analysis workflow. The interface is structured using a mode-action approach, where functionality is organized into context-specific operational modes.

The system provides four primary modes:

- **Plot Mode:** Enables detailed inspection of EEG signals for a single subject. Users can visualize raw or preprocessed signals to assess quality and identify patterns.
- **Grid Plot Mode:** Supports comparative visualization across multiple conditions or subjects. This mode facilitates identification of trends and differences at the cohort level.
- **Data Mode:** Provides structured access to dataset components, including metadata, event information, and preprocessing configurations. It serves as the primary interface for data preparation.
- **AI Mode:** Enables machine learning experimentation, including dataset construction, model training, and evaluation. Users can configure models and review performance metrics within this mode.

In addition to mode-based interaction, the system supports flexible subject selection through two approaches:

- **Single Subject Selection:** Users select an individual participant and task, allowing focused inspection and analysis.
- **Cohort Filtering (Meta Filter):** Users define filters based on attributes such as age range, sex, behavioral scores, or task performance. This enables large-scale exploratory analysis without manual subject selection.

Each mode exposes only the actions relevant to its context, reducing interface complexity and guiding users through the analysis workflow. Additionally, all user actions are logged to support transparency and reproducibility.

Screenshots and detailed interaction flows are shown in Chapter 5.

3.7 Target and Development System

3.7.1 Data Handling and Input Capabilities

- Load HBN-EEG from a local directory containing release folders (e.g., `cmi_bids_R*`) with subject directories `sub-*` and EEG files (`*_eeg.set`).
- Parse per-subject task events (`*_events.tsv`) and metadata needed for CCD behavioral targets.
- Support single-subject selection and cohort filtering using participant metadata and CCD-derived performance fields.
- Cache pipeline artifacts (filtered raw, epochs, evoked) on disk with a pipeline-version stamp to prevent accidental reuse after logic changes.

3.7.2 Core Processing and Modeling Requirements

- Consistent preprocessing with explicit parameters: band-pass, notch, resampling, channel selection, amplitude bounds, and cleaning thresholds.
- Deterministic epoch extraction for CCD trial-aligned windows (prestimulus/ITI), including label alignment validation.
- Dataset building for both ML feature vectors and DL epoch tensors.
- Training/testing pipelines with repeatable split strategies; group-aware splitting should be used when sufficient subjects exist.

3.7.3 User Interface and Visualization Needs

- Plots for signal inspection (time domain previews, PSD/evoked/topographic views, and task-specific verification such as SNR where applicable).

- Tables for inspection (events, channels/electrodes, metadata, and epoch summaries).

- Job outputs saved to disk in a consistent directory structure so the report can reference stable file artifacts.

3.7.4 Non-Functional Requirements

- **Reproducibility:** parameter DTOs (filter/epoch/train) and random seeds are saved in job specs; cached artifacts include a pipeline-version stamp.

- **Traceability:** every exported artifact (plots, datasets, models) can be traced back to a job directory containing a `spec.json` and logs.

- **Robustness:** the pipeline must fail fast on label/epoch alignment mismatches rather than silently producing mis-labeled training data.

- **Maintainability:** modular components (participant discovery, loader, processor, services, UI) with registry-driven action discovery.

3.8 Functional Requirements

Table 3.1 summarizes the functional requirements (FR) derived from the user stories and the implemented workbench design.

ID	Requirement
FR1	The system shall accept a user-configured dataset root directory and discover available releases, subjects, tasks, and runs.
FR2	The system shall provide two input schemas: (i) single subject selection, and (ii) cohort filtering by participant metadata and CCD-derived performance ranges.
FR3	The system shall load raw EEG and event tables for a selected task and expose basic inspection views (plots and tables).
FR4	The system shall apply preprocessing using explicit parameters (band-pass, notch, resample rate, channel selection, amplitude bounds, and cleaning thresholds).
FR5	The system shall build CCD trial-aligned epochs for a user-specified window (t_{min}, t_{max}) and shall validate label alignment deterministically.
FR6	The system shall cache intermediate artifacts (filtered raw, epochs, evoked) keyed by task/run and parameter hashes, and invalidate caches via a pipeline-version stamp.
FR7	The system shall build and export ML feature datasets as arrays `x`, `y`, `group` and store a dataset summary for inspection.
FR8	The system shall build and export DL epoch tensor datasets (e.g., $N \times 1 \times C \times T$) with aligned labels and group identifiers.
FR9	The system shall train/test models on saved datasets and export evaluation metrics and the trained model artifact with metadata.
FR10	The system shall support both inline execution (notebook output) and background execution as a job with persisted specification, logs, and outputs.

Table 3.1: Functional requirements summary for the EEG Workbench.

3.9 Non-Functional Requirements

ID	Requirement
NFR1	**Reproducibility:** A run shall be reproducible from a saved job specification (input schema + params DTOs + seed) without manual notebook edits.
NFR2	**Traceability:** Outputs shall be saved in a stable directory structure with logs so figures/tables in the report can be traced to a specific run.
NFR3	**Data integrity:** The system shall raise an error on detected trial/label alignment mismatches, and shall avoid silently producing shifted labels.
NFR4	**Performance:** The system should use caching and cohort batching to reduce redundant processing when iterating on parameters.
NFR5	**Usability:** The UI shall expose sensible defaults for key parameters (e.g., filter band, channel list) and group actions into understandable modes.
NFR6	**Maintainability/Extensibility:** New tasks (preprocessors), plots, and AI actions shall be addable through registration/DTO-driven mechanisms without rewriting the UI layer.

Table 3.2: Non-functional requirements summary.

3.10 Extensibility (Easy to Extend)

An explicit goal of the EEG Workbench is that extending the system requires adding *new modules* rather than modifying existing UI logic. This is achieved by three design choices:

- **Registry-driven actions (plug-in style):** plotting, ML, and DL capabilities are exposed as action registries where each new action registers a *name*, a *parameter DTO class*, and a *handler function*. The UI reads the controller's `spec` and automatically lists the new action.

- **DTO-driven parameters:** UI parameter panels are generated from dataclass DTOs (e.g., filter/epoch/train DTOs). Adding a new parameter set does not require hand-written widget layouts.

- **Task-specific preprocessors:** each dataset/task can provide its own preprocessing recipe via a lightweight registration mechanism (e.g., `register_preprocessor(task_name)`), so adding a new task does not require rewriting the pipeline core.

Table 3.3 summarizes typical extension scenarios and the minimal change surface expected.

Change case	**What to add**	**Why it is low effort**
EC1: Add a new plot	A new plot function + a params DTO (optional) registered into the plot action registry.	The UI reads the updated `spec` and auto-generates parameter widgets from the DTO; no UI refactor is required.
EC2: Add a new ML experiment	A new ML action registered via a decorator (name + DTO + handler) that consumes saved datasets.	Training/testing actions share common dataset discovery and job output conventions, so new actions reuse the same execution and export path.
EC3: Add a new DL model training	A new DL action with its own training DTO (batch size, epochs, learning rate, etc.) and handler.	DL actions are isolated behind the DL service; registering the action makes it available immediately in the same UI mode.
EC4: Support a new HBN task	A new task-specific preprocessor registered under a task name, including event parsing and epoch definitions.	The pipeline core calls a registered preprocessor per task; adding a new task does not require changing unrelated tasks or UI components.

Table 3.3: Extensibility change cases showing how new capabilities can

3.11 Traceability (Stories to Requirements)

User story	Mapped requirements
US1	FR1, FR2
US2	FR3, FR4
US3	FR4, FR6, NFR1
US4	FR5, NFR3
US5	FR7, FR8, FR6
US6	FR9, NFR2
US7	FR10, NFR2

Table 3.4: Traceability matrix from user stories to requirements.

Chapter 4

Software Architecture Design

This chapter presents the software architecture that supports reproducible EEG experimentation in this project. It describes a layered architecture, the main execution flow, the AI processing pipeline, and the key data schemas used to make runs traceable and repeatable.

4.1 Software Architecture

The system is designed using a **layered client–server architecture**, consisting of a React-based frontend and a FastAPI backend. The backend is implemented as a modular monolith, where functionalities are organized into domain-specific modules such as API handling, data processing, visualization, and machine learning.

At its core, the system follows a **pipeline-oriented architecture**, where EEG data is processed through sequential stages (e.g., preprocessing, epoch extraction, and feature generation). This structure enables reproducibility and flexible recomputation when parameters change.

4.2 Design Patterns

To ensure a modular and extensible architecture capable of handling high-throughput data, the following design patterns were implemented:

- **Pipeline Pattern:** EEG processing workflow
- **Facade Pattern:** API endpoints
- **Strategy Pattern:** parameter-driven behavior

- **Factory Pattern:** model instantiation
- **Registry Pattern:** task preprocessor selection
- **Observer Pattern:** WebSocket progress updates
- **Repository Pattern:** structured EEG cache
- **Composite Pattern:** Pipeline Session is a combination of schemas

4.3 Domain Model

The domain model of the proposed EEG analysis platform represents the key entities and their relationships within the system. The design focuses on supporting a structured workflow for EEG data analysis and machine learning experimentation.

The central entity in the system is the **Dataset**, which represents EEG data organized in BIDS format. Each dataset consists of multiple **Subjects**, where each subject contains EEG recordings and associated metadata for specific tasks.

The **User** interacts with the system to perform analysis operations on the dataset. These operations include signal inspection, preprocessing, dataset construction, and model training.

A **PipelineSession** entity represents the sequence of preprocessing operations applied to EEG data. This includes filtering, resampling, artifact removal, and epoching. The pipeline operates on raw EEG signals and produces processed outputs such as prefiltered signals, cleaned data, and epochs.

To improve efficiency, the system incorporates a caching mechanism that stores intermediate results. These cached outputs are associated with specific subjects and parameter configurations, allowing the system to reuse previously computed results and reduce redundant processing.

The **AI Model** entity represents machine learning models used for experimentation. Models can be trained using processed datasets and evaluated using standard performance metrics.

Overall, the domain model reflects a modular structure where data processing, visualization, and machine learning components are integrated into a unified workflow.

4.4 Design Class Diagram

The system architecture follows a layered design, where different components are organized based on their responsibilities. The main modules correspond to the backend structure implemented using FastAPI.

At the top level, the **API layer** (app/api/endpoints) handles incoming requests from the frontend. It defines endpoints for operations such as data loading, preprocessing, visualization, and model training. This layer acts as the interface between the user interface and the backend system.

The **Schema/DTO layer** (app/schemas) defines structured data models used for request validation and response formatting. This includes parameter schemas for preprocessing configurations and UI-related data representations.

The **Core module** (app/core) manages configuration and dataset loading. It is responsible for accessing EEG data from BIDS-formatted directories and preparing it for further processing.

The **Pipeline module** (app/pipeline) implements the core EEG processing logic. It receives raw EEG data and applies preprocessing steps such as filtering, resampling, and epoching. This module also interacts with the caching mechanism to store and retrieve intermediate results.

The **Visualization module** (app/plots) contains submodules for different types of outputs:

- **plot:** single-view signal visualization
- **grid plot:** comparative visualization
- **data:** tabular data summaries
- **ai:** model-related results and metrics

These modules generate outputs that are returned to the frontend for display.

The **AI Model module** (app/ai_models) defines machine learning models used within the platform. These models can be trained and evaluated using processed datasets.

Additionally, the system maintains a **job/output storage structure** (jobs/ai/train/...) that stores training results, logs, and model artifacts, along with the cache for each pipeline step in **eegcache** folder (such as prefiltered, cleaned, epochs, etc.) While not a full asynchronous job system, this structure supports persistence and traceability of preprocessing and model training runs.

This modular design ensures separation of concerns, making the system maintainable, extensible, and aligned with the research workflow.

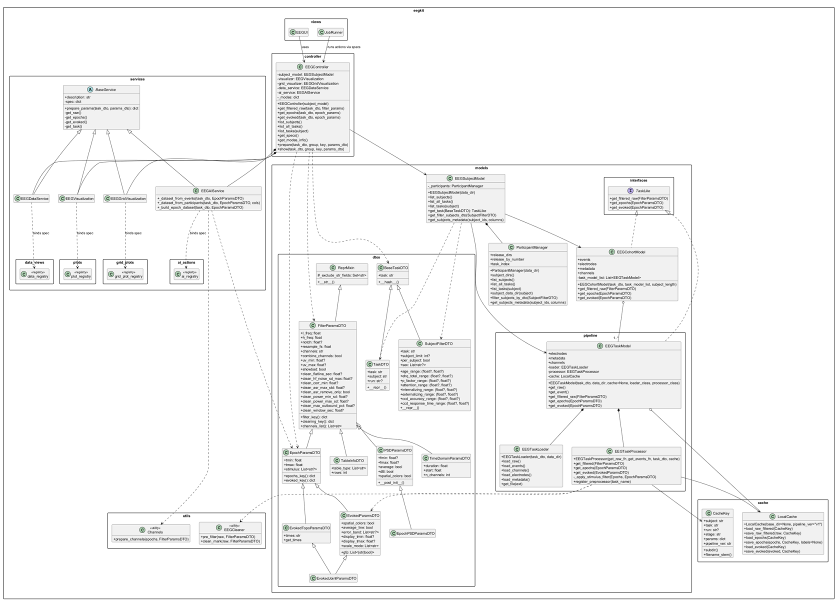

Figure 4.1: System architecture overview

4.5 Sequence Diagram

4.5.1 Experiment Execution Flow

Scenario: Run an experiment

1. Select subjects/sessions and the CCD task.
2. Load BIDS/HED events and construct CCD-ITI windows with aligned RT and correctness labels.
3. Apply a fixed preprocessing configuration to the ITI epochs.
4. Compute verification plots (e.g., SNR spectra) to confirm steady-state characteristics.
5. Extract features and train models for RT regression and correctness classification.
6. Evaluate using standard metrics and export a run summary (configs, metrics, and plots).

4.6 Pipeline

4.6.1 AI Processing Pipeline

The AI processing pipeline is designed to enforce traceability. Each stage consumes explicit inputs and produces versioned outputs:

1. **Ingestion:** load CCD runs and parse events/metadata.[1]
2. **Trial construction:** extract fixed-length ITI windows and attach labels.
3. **Preprocessing:** filter, normalize, mitigate artifacts, and create epochs.
4. **Verification:** compute SNR spectra and related plots for steady-state validation.

[1] [2].

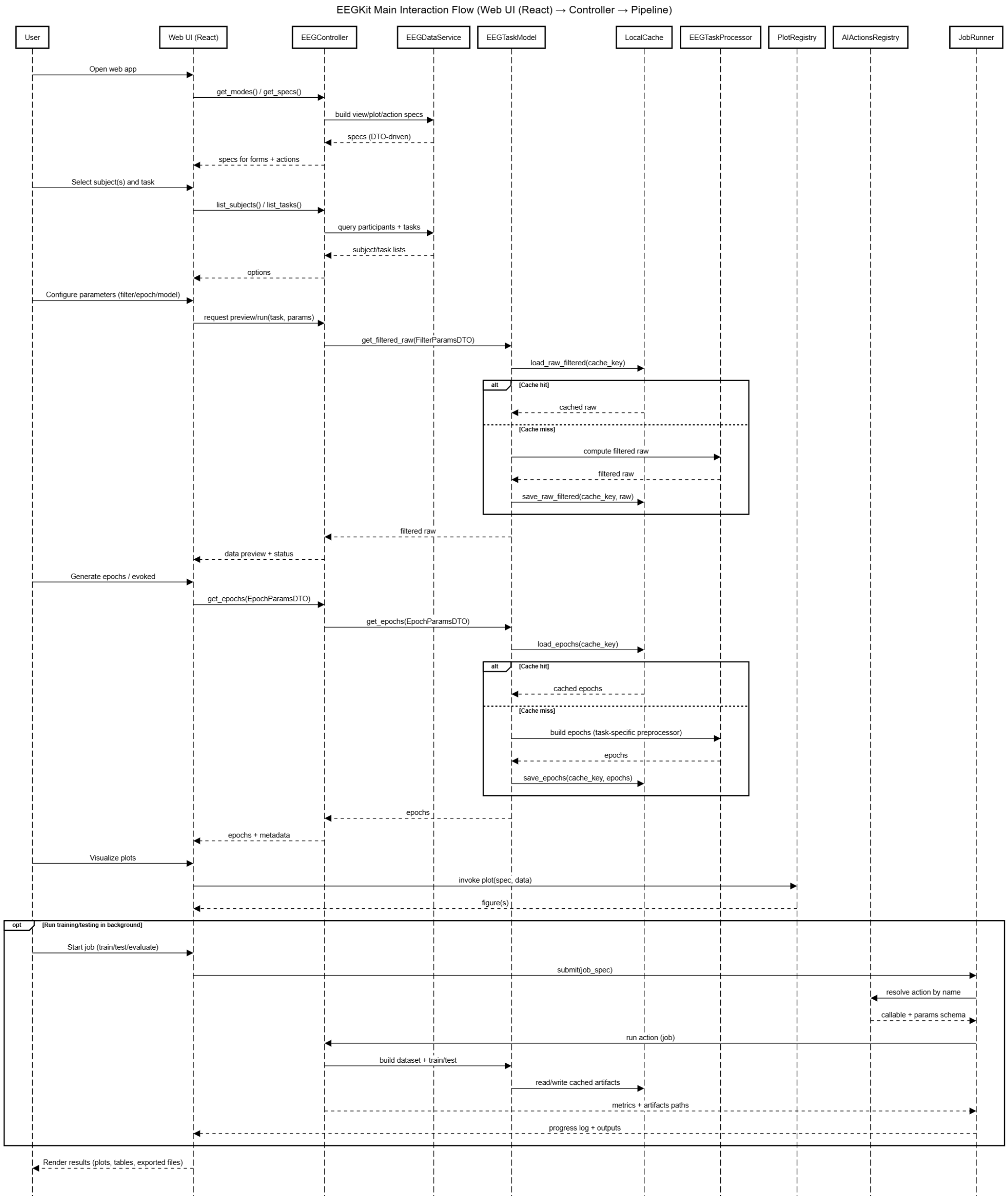

Figure 4.2: EEGKit main interaction flow (Web UI (React) → Controller → Pipeline)

5. **Features:** compute ITI-derived features (e.g., spectral power at stimulation frequencies/harmonics).

6. **Modeling:** train RT regressors and correctness classifiers.

7. **Evaluation:** compute metrics and export artifacts for reporting.

Chapter 5

AI Component Design

This chapter describes how the AI component is integrated into the overall system workflow, from dataset construction and signal verification to model training and evaluation. It follows an AI-canvas-style requirement analysis, documents model development and evaluation, and describes how the user experience and deployment support reproducible experiments.

5.1 Business Context and AI Integration

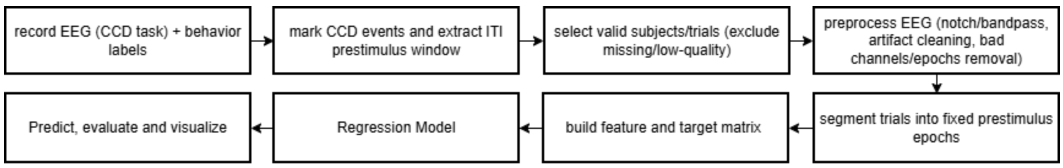

Figure 5.1: Workflow for prestimulus EEG-based behavioral prediction from CCD-ITI segments.

Diagram Explanation: The system is designed as an experimental EEG research platform integrating signal visualization, dataset inspection, and machine learning experimentation into a unified workflow. The frontend interface allows users to explore EEG signals, construct datasets, and execute AI models, while the backend handles preprocessing, model execution, and evaluation.

The workflow proceeds as:

signal inspection → dataset verification → model training → evaluation

This ensures that model results are directly connected to verified signal characteristics rather than blindly training on raw data.

Why is AI suitable for this problem?

EEG analysis involves high-dimensional temporal signals with complex spatial relationships across electrodes. Manual feature engineering is difficult because:

- patterns vary across subjects
- noise and artifacts are common
- signal distributions change across recording sessions

Machine learning models can learn robust mappings from EEG-derived features to behavioral targets more effectively than fixed heuristics.

Is the problem large, complex, or always changing?

Yes.

EEG data presents three major challenges:

- Large — continuous multi-channel time series
- Complex — spatial-temporal correlations
- Non-stationary — signals differ between subjects and sessions

Because of this variability, deterministic rule-based systems are unsuitable.

Can we accept an answer that's not 100% perfect?

Yes. This is a research system.

The system is intended for research experimentation rather than safety-critical decision making. Model predictions are used as analytical indicators, not medical diagnoses. Therefore approximate predictions with measurable error are acceptable.

5.2 Goal Hierarchy

Organizational Goal: Enable efficient experimentation in EEG machine learning research

System Goal: Provide an interactive platform that links signal inspection with machine learning experimentation

User Goal: Explore EEG signals, construct datasets, train models, and compare results without manual scripting

AI Model Goal: Learn mappings that generalize across subjects for RT prediction (regression) and correctness prediction (classification)

Success Metrics:

- reduced prediction error (e.g., MAE/MSE) for RT
- improved correctness prediction (accuracy, F1-score, precision, recall)
- consistent performance on unseen subjects
- reduced experiment setup time
- reproducible experiment workflow

5.3 Task Requirements Analysis Using AI Canvas

5.3.1 AI Task Requirements

- **Requirements (REQ):** Predict behavioral outcomes from prestimulus CCD-ITI EEG segments in HBN-EEG.
- **Specifications (SPEC):** Evaluate models that map CCD-ITI features to RT (regression) and trial correctness (classification).
- **Environment (ENV):** Evaluated on multi-subject EEG datasets with different recording conditions. (e.g., noise, subject variation).

5.3.2 AI Canvas Summary

- **Input:** Preprocessed EEG segments
- **Output:** Predicted RT (regression) and predicted correctness (classification)
- **Success Criteria:** Lower RT prediction error and higher correctness classification performance with stable generalization to unseen subjects.

5.3.3 Innovation

The system integrates visualization-driven dataset validation with model training inside a single interactive interface, reducing mismatch between inspected signals and training data.

5.4 AI Model Development and Evaluation

5.4.1 Model Development and Training Strategy

The modeling workflow is designed to be repeatable and target-aware:

- **Inputs:** preprocessed CCD-ITI epochs and ITI-derived feature tables.
- **Targets:** RT (regression) and correctness (binary classification).
- **Baseline-first approach:** start with simple, interpretable models before considering more complex architectures.
- **Split strategy:** prefer subject-independent splits to evaluate generalization and reduce overly optimistic estimates.

5.4.2 Testing and Validation Approach

Validation focuses on preventing misleading results caused by data issues or leakage:

- sanity checks for trial counts, label distribution, and missing channels,
- verification plots (e.g., SNR spectra) to confirm steady-state characteristics,
- consistent preprocessing parameters and fixed random seeds,
- run logging of dataset filters, split definitions, and model configurations.

5.4.3 Performance Results and Justification

Model results are reported using target-appropriate metrics:

- **RT regression:** MAE, MSE, and and coefficient of determination (R^2)

- **Correctness classification:** accuracy, F1-score, precision, and recall.

- **Training dynamics:** training loss and validation loss across epochs

These metrics are generated during model training and presented through training logs and visualizations, allowing users to monitor both model accuracy and generalization performance.

The use of validation-based metrics ensures that model performance is assessed on unseen data, supporting reliable evaluation and reducing the risk of overfitting.

5.5 User Experience Design with AI

The platform follows a research-oriented workflow where users first inspect signals before executing machine learning models. To support this process, the interface uses a **mode-action** structure, and additionally provides flexible **subject selection and filtering** to accommodate both single-subject inspection and cohort-level analysis.

Interface Overview: The interface contains four primary operational modes:

- **Plot Mode:** — detailed single-view signal inspection
- **Grid Plot Mode:** — comparative multi-condition visualization
- **Data Mode:** — structured data inspection and preparation
- **AI Mode:** — machine learning experimentation

Each mode exposes only the actions relevant to the selected context, and all user actions are logged to support transparency and reproducibility.

- **Subject Selection and Filtering** Users can choose between two primary input types:
  - **Single Subject:**
    - User selects an individual participant and task.
    - All mode actions are executed for the selected subject only.
    - This mode is ideal for detailed inspection or troubleshooting of individual EEG recordings.
  - **Cohort Filter (Meta Filter):**
    - User defines filters across attributes: sex, age range, attention, internalizing/externalizing scores, CCD accuracy, response time.
    - Optional "Per Subject" toggle for applying filters individually or collectively.
    - This mode enables exploratory data analysis and large-scale EEG studies without manual participant selection.

Visualization and Inspection Modes

5.5.1 Plot Mode

Provides detailed EEG inspection including sensor layout, time-domain plots, frequency plots, epoch plots, evoked responses, and SNR analysis.

Available actions include:

- sensor layout visualization
- time-domain signal plots
- frequency-domain plots
- epoch visualization
- evoked response plots
- evoked topography plots
- SNR spectrum analysis

5.5.2 Grid Plot Mode

Allows comparison across conditions using PSD, SNR, and evoked grids.

Available actions include:

- PSD Grid
- SNR Grid
- Evoked Grid

5.5.3 Data Mode

Displays structured EEG data tables to confirm dataset composition.

Available actions include:

- EEG sample tables

- epoch tables
- metadata

5.5.4 AI Interaction Mode

AI Mode enables machine learning experimentation using the prepared EEG data. The interface provides a set of interrelated actions that support an iterative workflow from dataset construction to model training and analysis.

Available actions include:

- Build Dataset
- TrainEEGNetMultiRegression
- Model Summary

System Behavior: These actions are interdependent: dataset construction is required before training, and the model summary reflects the structure of the model. Performance metrics are generated during training and updated dynamically.

Logs and outputs are available for inspection and export, supporting transparency and reproducibility.

This design emphasizes transparency by exposing intermediate training metrics directly through logs rather than abstracted summaries.

Feedback Loop: inspect → adjust → train → evaluate → compare → refine

Researchers can iteratively adjust dataset configurations,rerun experiments, and retrain models without rewriting scripts or managing computing environments. This significantly shortens the experimental cycle compared to notebook-based pipelines.

Interface Screenshots:

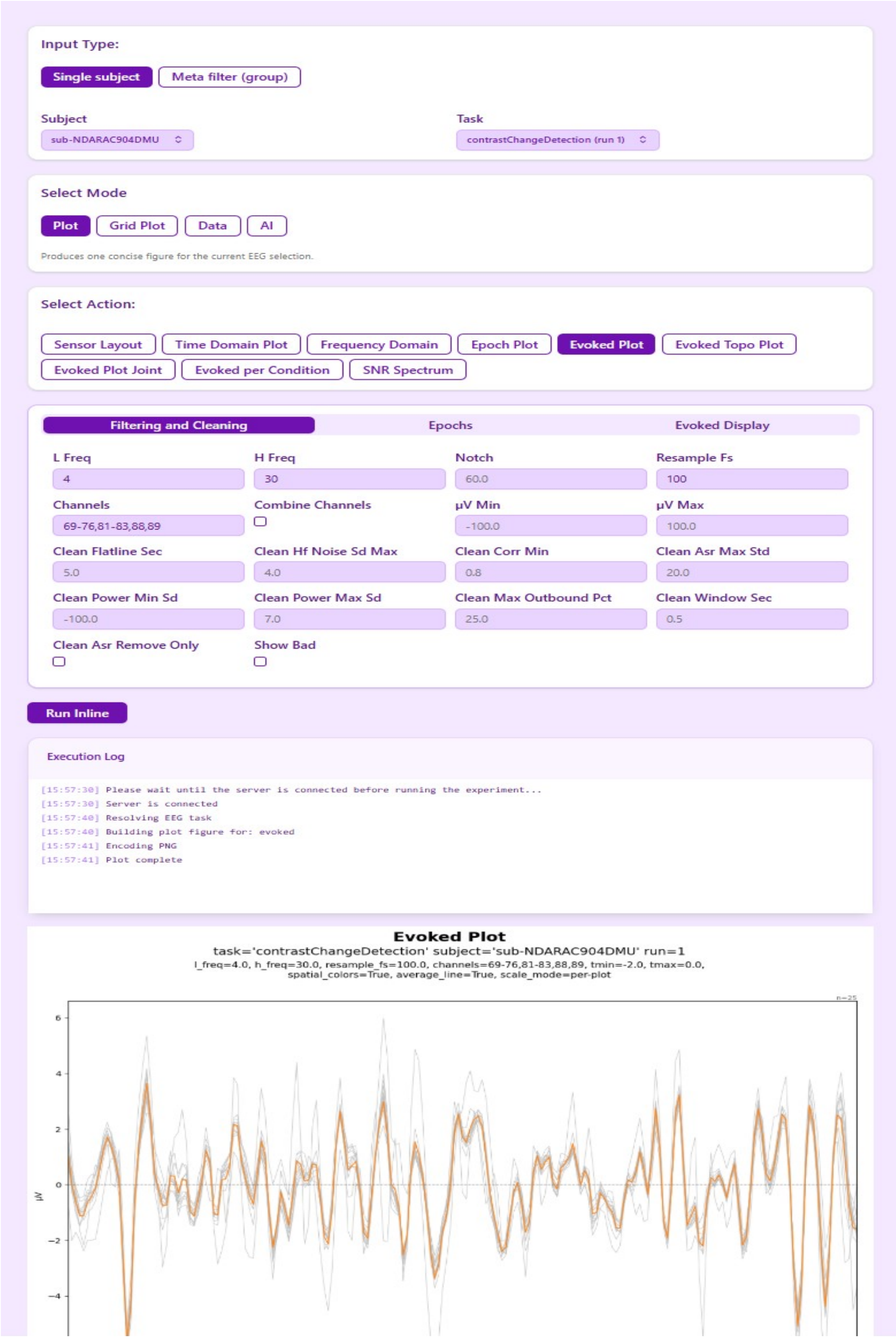

Figure 5.2: React Notebook Interface – Model visualization and execution for single subject

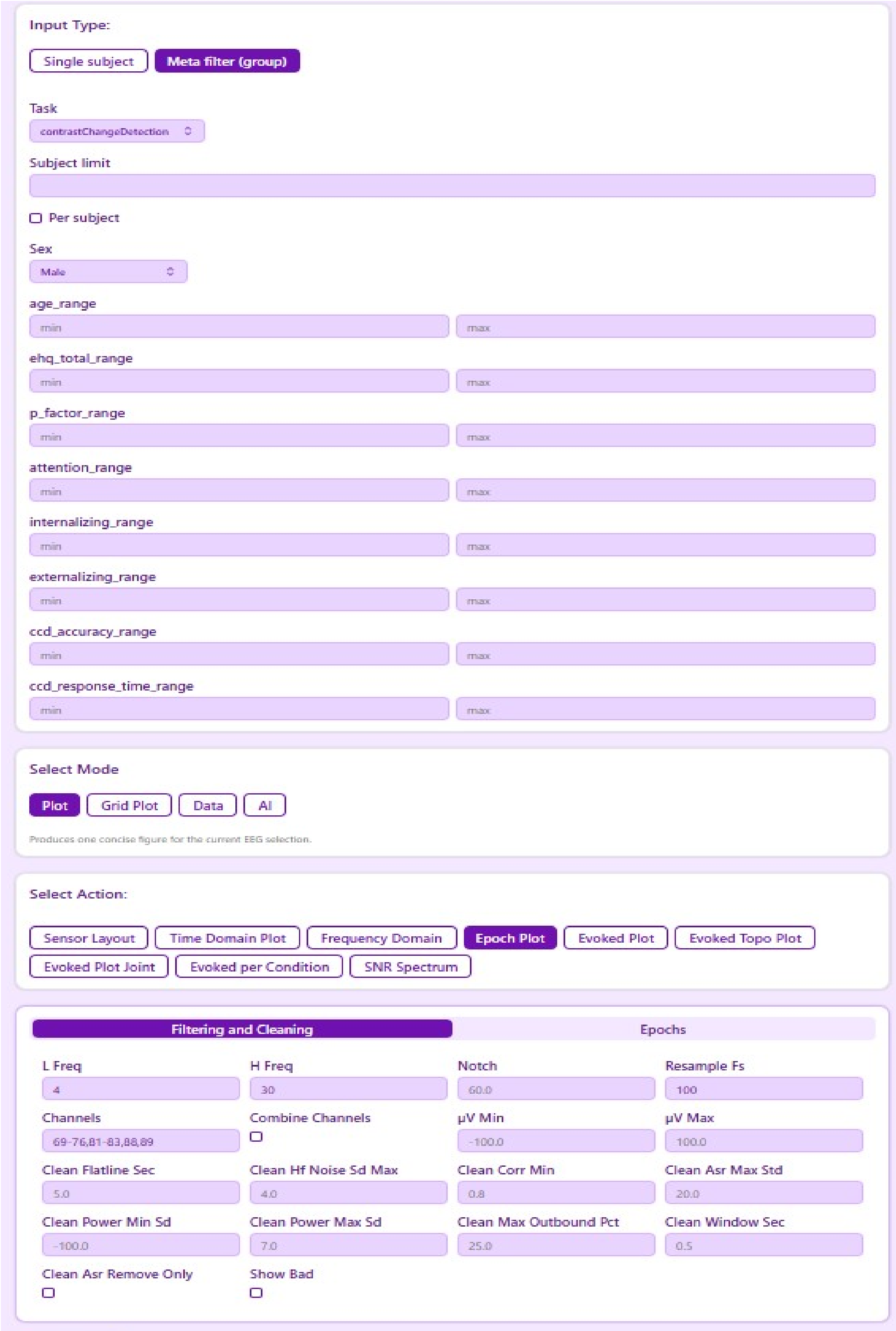

Figure 5.3: React Notebook Interface – Model visualization and execution for multi subject

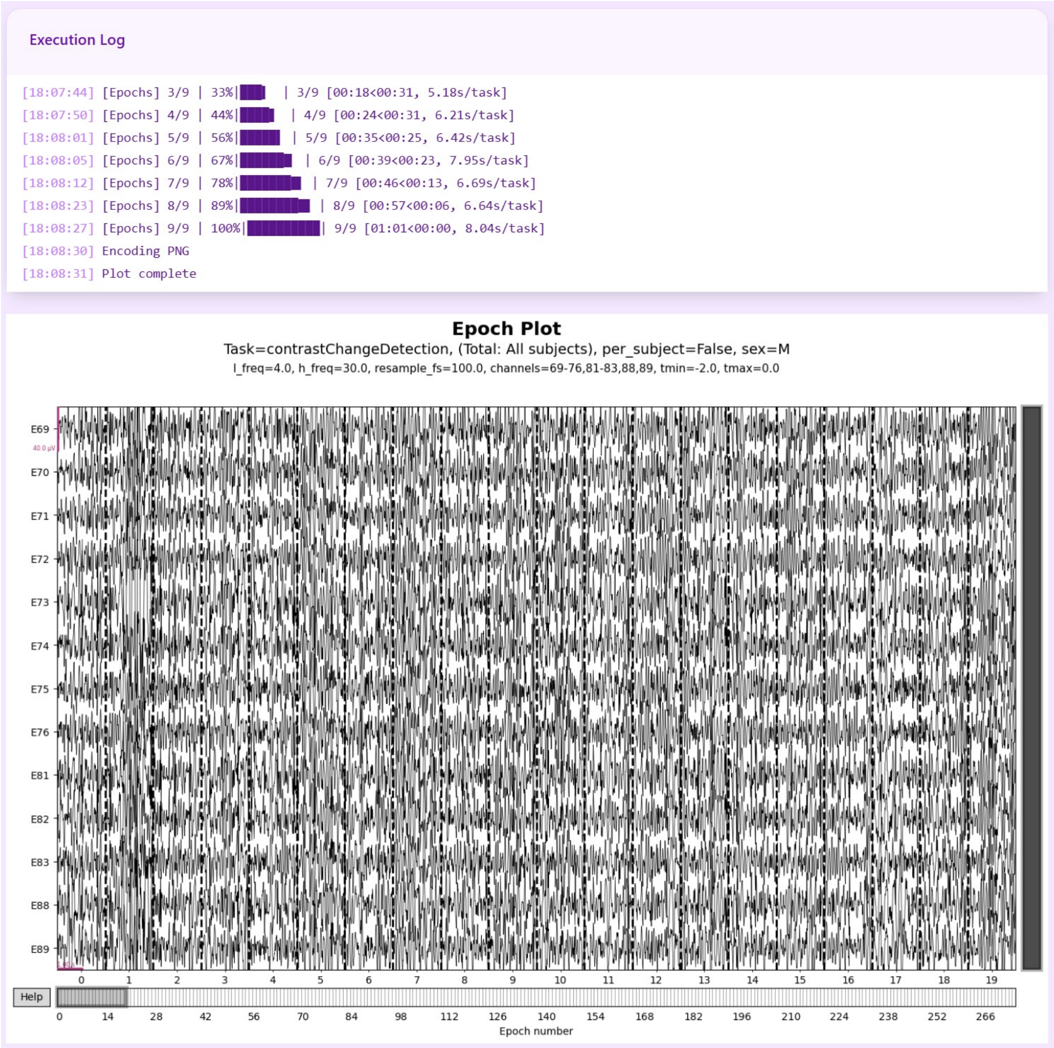

Figure 5.4: React Notebook Interface – Model visualization and execution for multi subject (2)

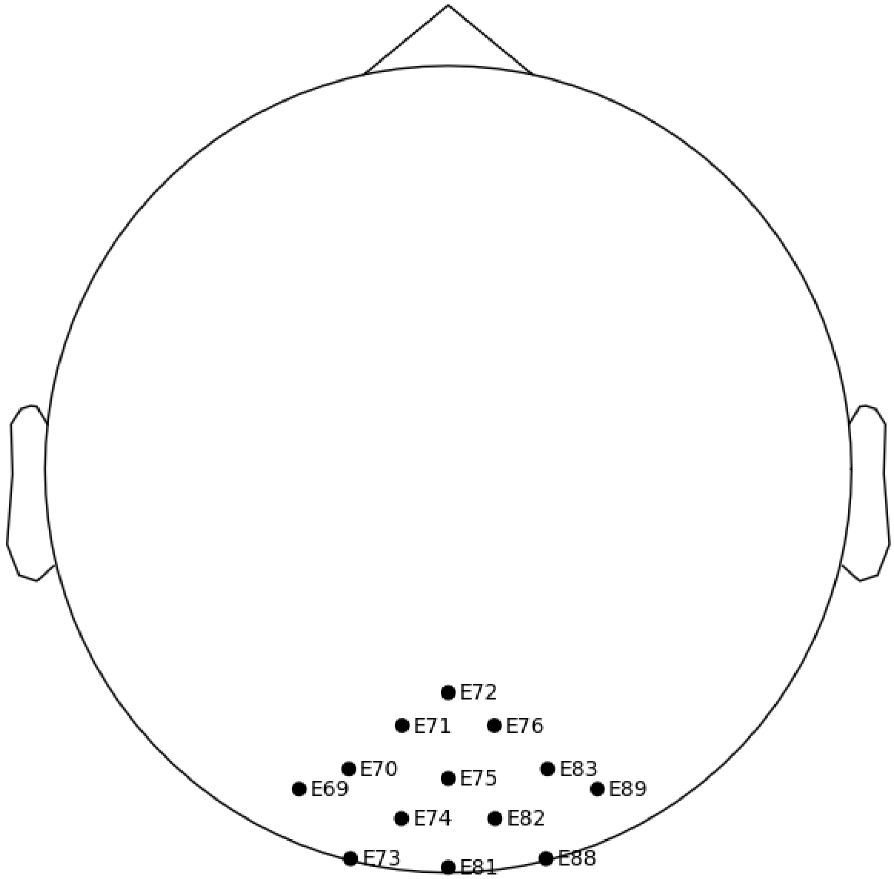

Figure 5.5: Sensor layout

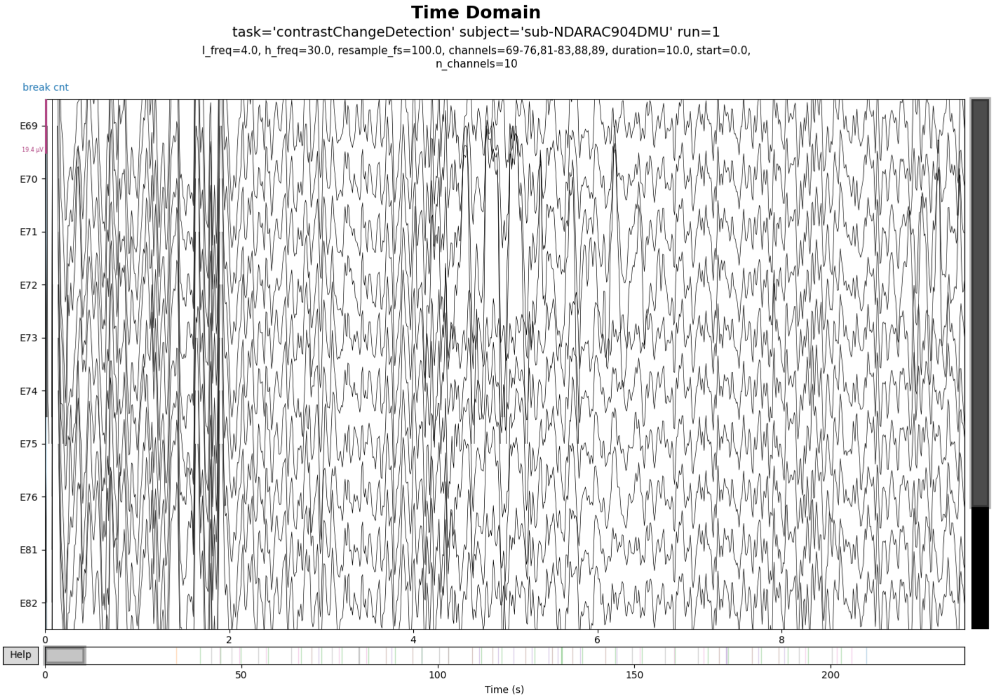

Figure 5.6: Time-domain signal

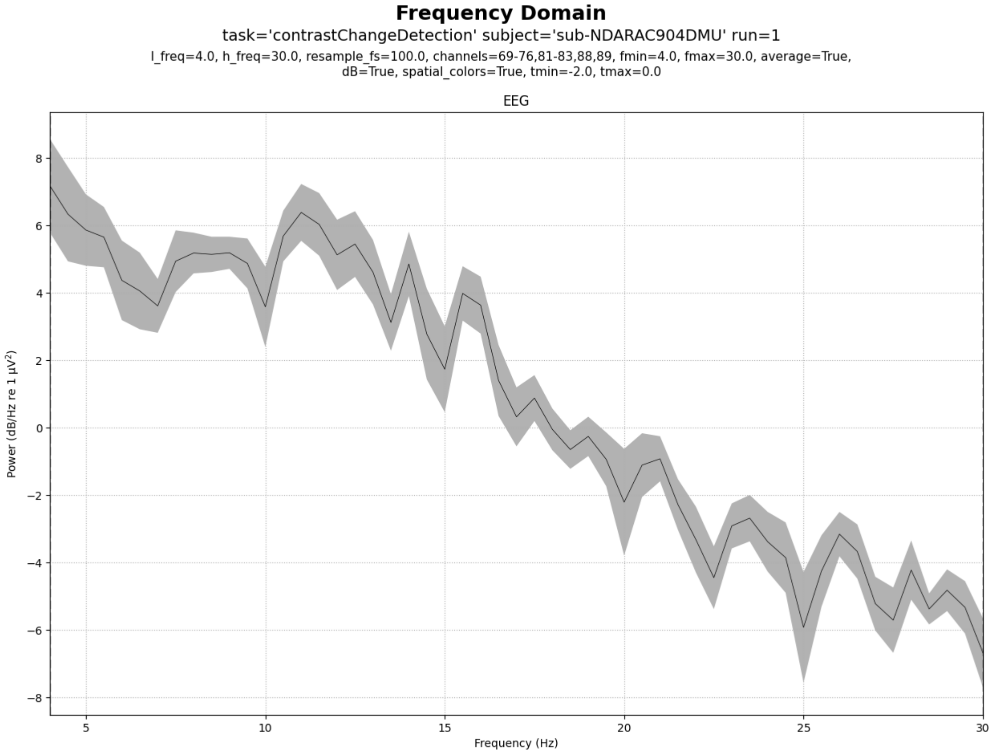

Figure 5.7: Frequency spectrum

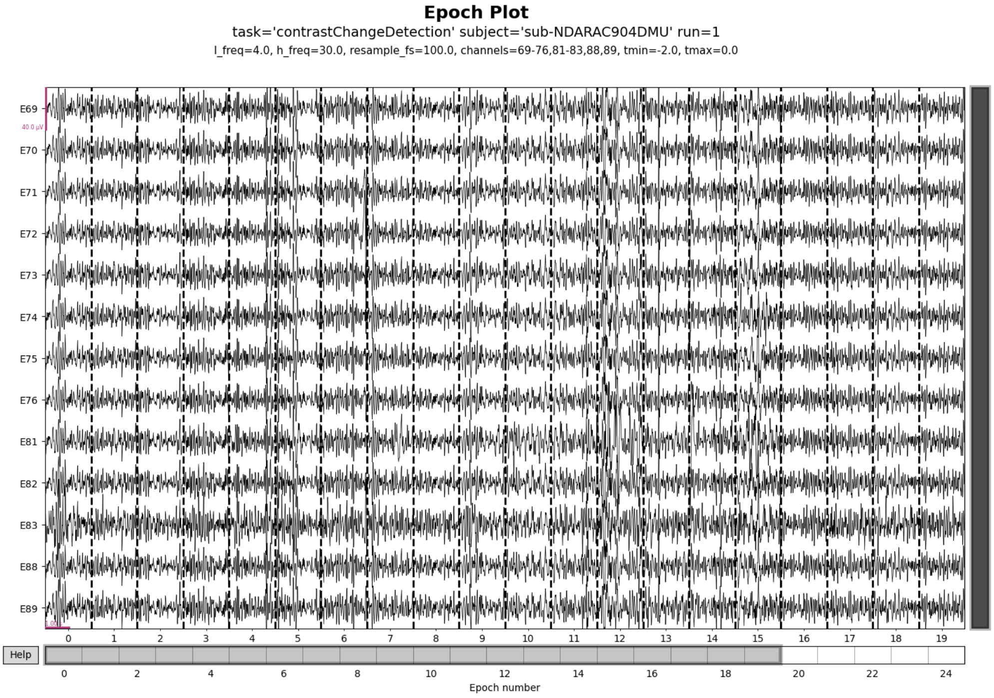

Figure 5.8: Epoch visualization

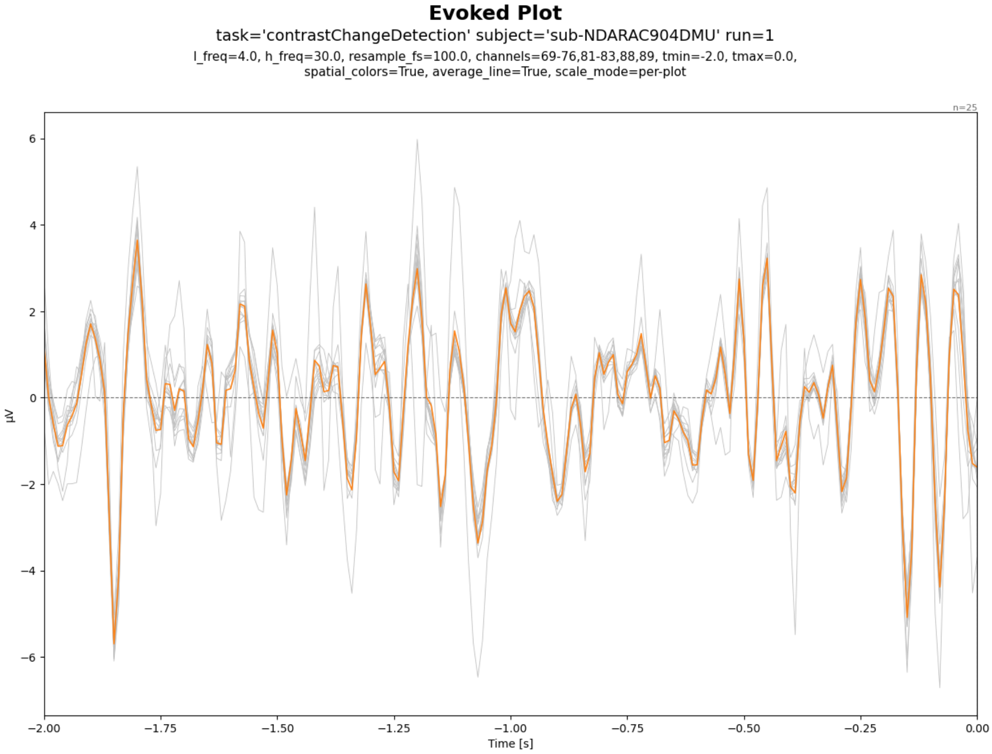

Figure 5.9: Evoked response

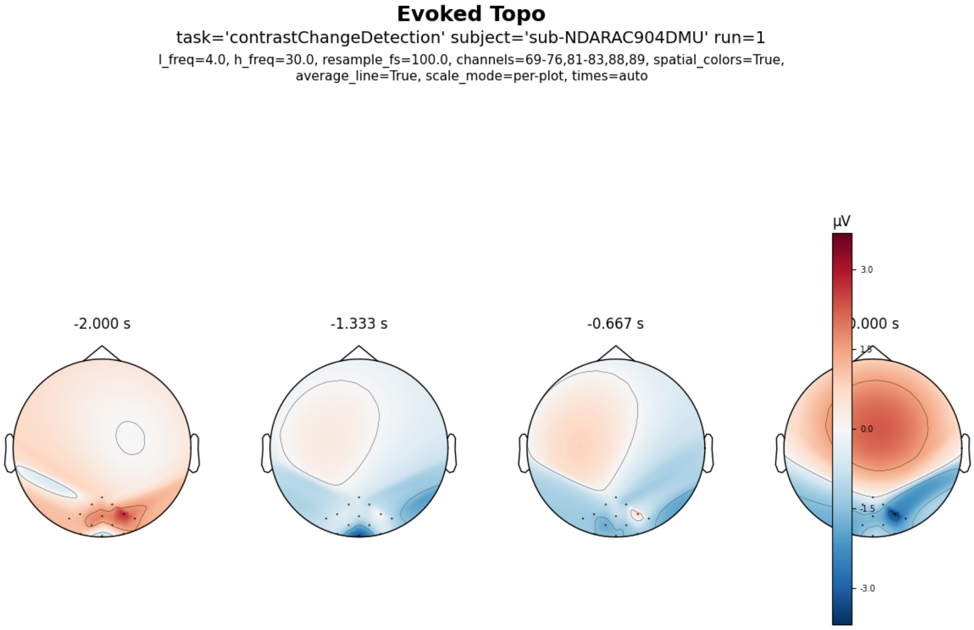

Figure 5.10: Evoked topomap

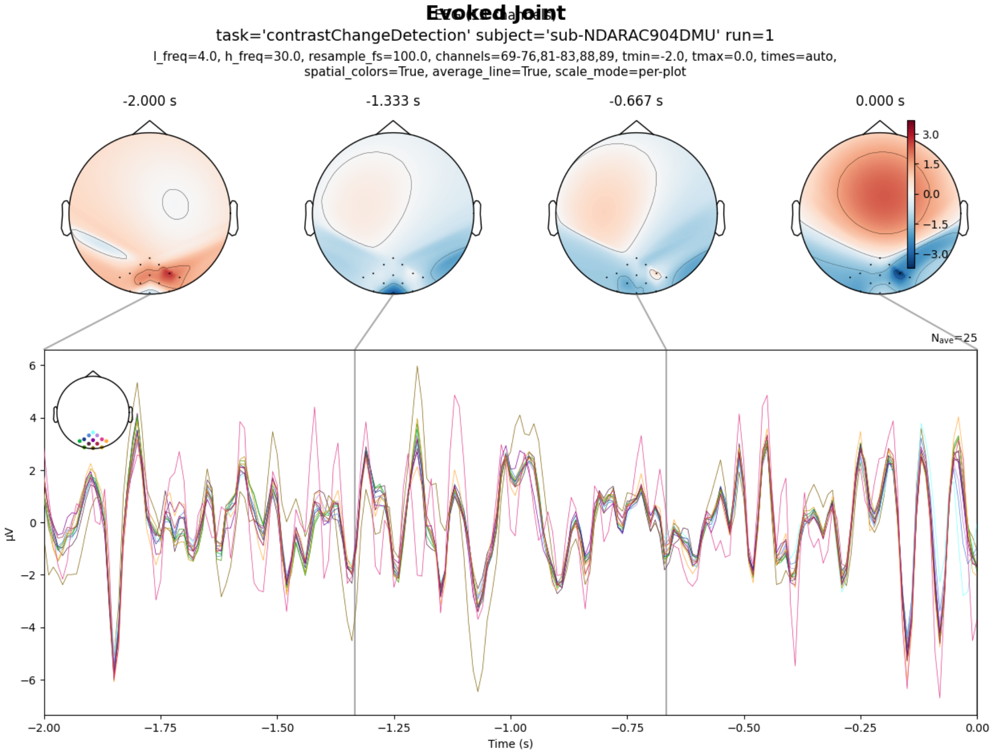

Figure 5.11: Evoked joint plot

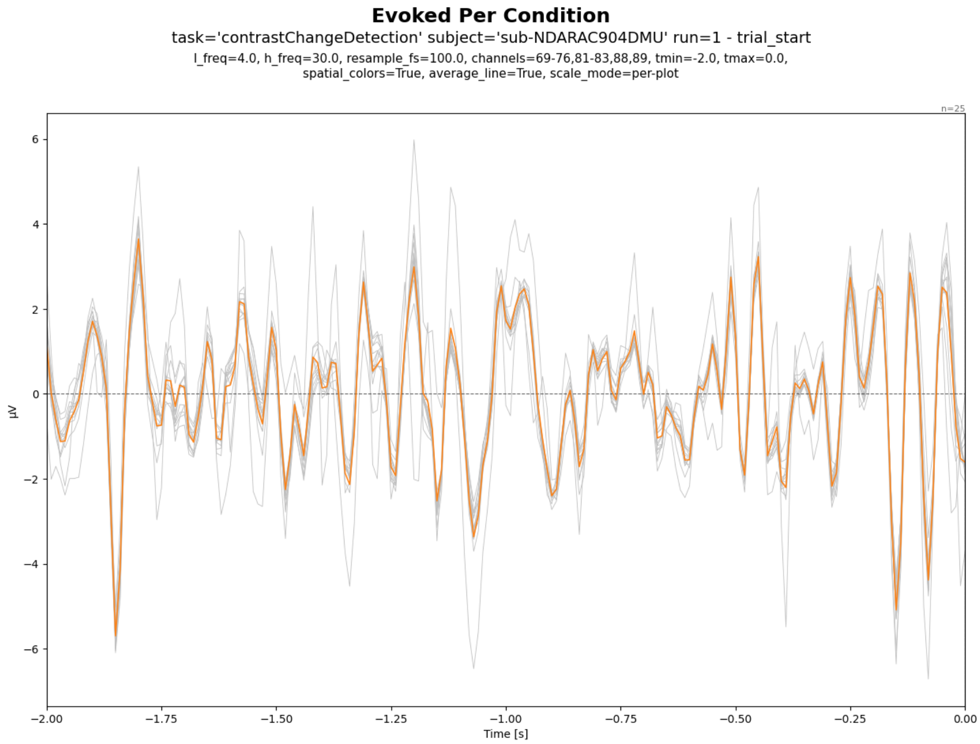

Figure 5.12: Evoked per condition

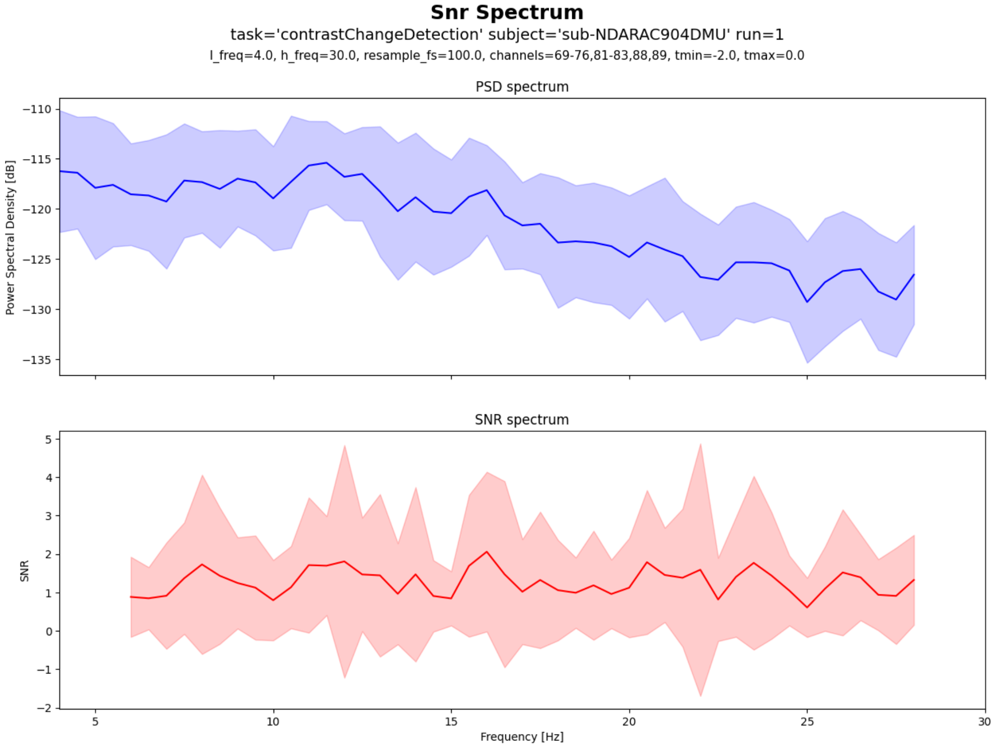

Figure 5.13: Signal-to-noise ratio spectrum

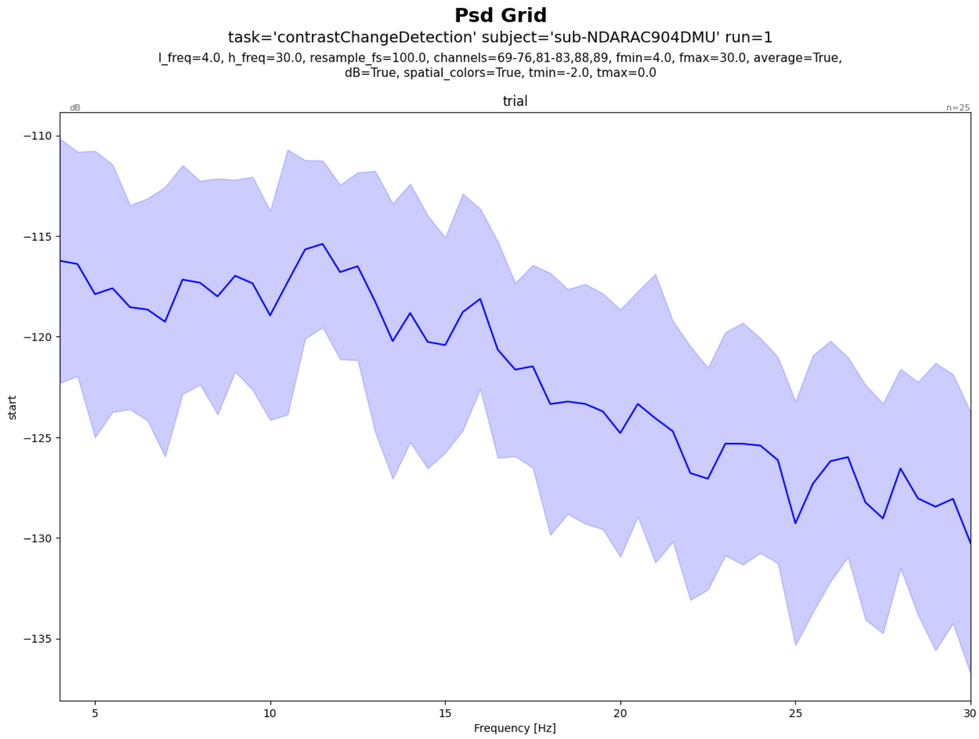

Figure 5.14: Power spectral density grid

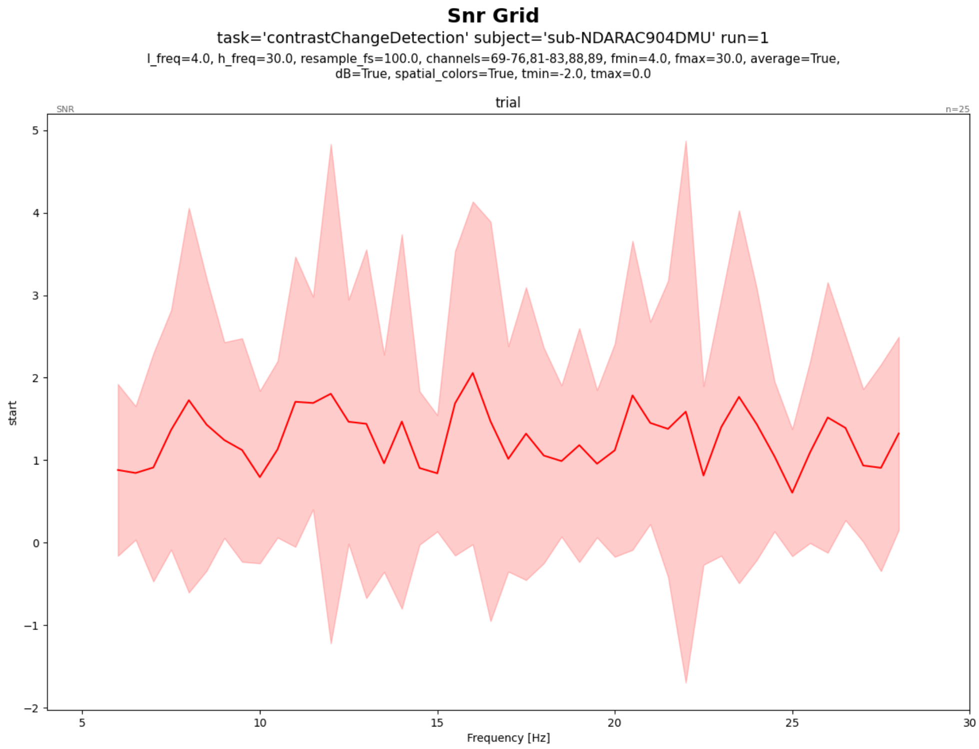

Figure 5.15: Signal-to-noise ratio grid

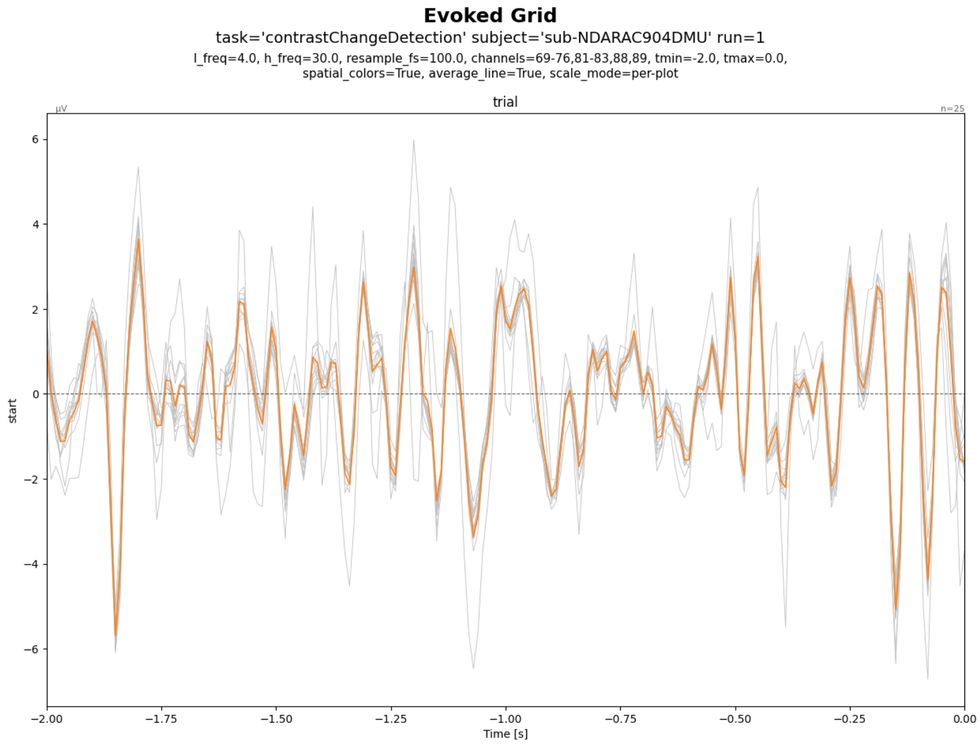

Figure 5.16: Evoked grid view

onset	duration	sample	value	event_code	feedback
0		0	break cnt	break cnt	
33.400		16,700	contrastChangeB1_start	94	
39.484		19,742	contrastTrial_start	5	
42.284		21,142	right_target	9	
44.414		22,207	right_buttonPress	13	smiley_face
44.684		22,342	contrastTrial_start	5	
47.484		23,742	right_target	9	
49.444		24,722	right_buttonPress	13	smiley_face
49.884		24,942	contrastTrial_start	5	
54.284		27,142	right_target	9	

Figure 5.17: EEG Table

label	n_epochs	n_channels	timespan_sec	sampling_rate	duration_per_epoch_sec
trial_start	25	128	2	100	2.010

Figure 5.18: Epochs Table

field	value
PowerLineFrequency	60
TaskName	contrastChangeDetection
EEGChannelCount	129
EEGReference	Cz
RecordingType	continuous
RecordingDuration	234.104
SamplingFrequency	500
SoftwareFilters	n/a

Figure 5.19: Metadata Table

n_epochs	n_channels	n_times	sfreq	label::93.0556
25	13	201	100	25

Figure 5.20: Build Dataset Table

epoch	train_loss	val_loss	val_mae	val_r2
1	8785.288	8644.565	92.976	—
2	7657.540	8628.153	92.888	—
3	7029.752	8612.935	92.806	—
4	6579.648	8599.336	92.733	—
5	6160.696	8579.167	92.624	—
6	5748.555	8556.205	92.500	—
7	5390.962	8517.980	92.293	—
8	4998.942	8471.551	92.041	—
9	4629.825	8418.859	91.754	—
10	4231.050	8354.287	91.402	—
11	3862.044	8270.908	90.945	—
12	3482.110	8162.792	90.348	—
13	3158.336	8036.194	89.645	—
14	2826.153	7879.874	88.769	—
15	2488.775	7690.543	87.696	—
16	2187.789	7486.270	86.523	—
17	1870.673	7235.068	85.059	—
18	1601.797	6951.105	83.373	—
19	1377.668	6628.659	81.417	—
20	1162.443	6287.875	79.296	—
21	964.768	5937.247	77.054	—
22	750.151	5559.127	74.560	—
23	605.925	5176.067	71.945	—
24	465.719	4776.360	69.111	—
25	356.632	4384.486	66.215	—
26	277.896	3997.530	63.226	—
27	189.174	3634.381	60.286	—
28	133.263	3285.107	57.316	—
29	84.388	2969.587	54.494	—
30	45.871	2679.816	51.767	—
31	26.688	2415.238	49.145	—
32	18.966	2180.149	46.692	—
33	13.227	1955.059	44.216	—
34	14.910	1780.155	42.192	—
35	25.970	1598.232	39.977	—
36	16.541	1450.283	38.082	—
37	28.291	1308.515	36.172	—
38	28.556	1169.425	34.197	—
39	43.122	1056.393	32.501	—
40	43.380	959.707	30.978	—
41	46.487	869.578	29.487	—
42	39.488	766.095	27.677	—
43	55.277	680.290	26.080	—
44	33.083	620.142	24.900	—
45	32.417	539.505	23.225	—
46	28.597	490.384	22.142	—
47	23.332	427.502	20.675	—
48	12.073	383.657	19.584	—
49	11.887	339.250	18.417	—
50	10.848	305.535	17.474	—

Figure 5.21: Train EEGMultiNetRegression Table

Layer	Output Shape	Params
EEGNetMultiReg	[1, 1]	100,977
Conv2d	[1, 16, 13, 138]	1,040
BatchNorm2d	[1, 16, 13, 138]	0
ZeroPad2d	[1, 16, 14, 171]	0
Conv2d	[1, 32, 13, 140]	32,800
BatchNorm2d	[1, 32, 13, 140]	0
MaxPool2d	[1, 32, 6, 35]	0
ZeroPad2d	[1, 32, 13, 38]	0
Conv2d	[1, 64, 6, 35]	65,600
BatchNorm2d	[1, 64, 6, 35]	0
MaxPool2d	[1, 64, 3, 8]	0
Linear	[1, 1]	1,537
TOTAL PARAMS		100,977
TRAINABLE PARAMS		100,977
NON-TRAINABLE PARAMS		0
TOTAL MULT-ADDS (M)		75.340
INPUT SIZE (MB)		0.010
FWD/BWD SIZE (MB)		0.800
PARAM SIZE (MB)		0.400

Figure 5.22: Model Summary Table

5.6 Deployment Strategy

5.6.1 Deployment Plan

The AI component is deployed in a research environment using both local execution and cloud notebooks.

The system consists of:

- Frontend — interactive research interface
- Backend — REST API service
- Model — behavioral prediction model

Technologies used:

- React — interactive frontend framework

- FastAPI — model service endpoints
- TensorFlow — neural network execution
- scientific Python libraries — preprocessing and evaluation

The frontend web system replaces manual notebook execution and allows controlled interaction with the EEG pipeline, and the backend exposes endpoints for training, prediction, and evaluation, allowing modular integration and testing.

The model in our system targets prestimulus behavioral prediction from CCD-ITI segments by mapping EEG-derived features to reaction time (RT) (regression) and trial correctness (classification).

5.7 Proof of Concept (Service Demonstration)

The pipeline can be exercised end-to-end under a subject-independent evaluation setting. In this setting, models are trained on a group of subjects and tested on unseen individuals, which better reflects generalization and avoids overly optimistic estimates from within-subject testing.

The system supports the workflow:

visualization → dataset selection → training → inference → evaluation

Results can be accessed through the interface and (when enabled) through API endpoints.

5.8 Reflection and Future Development

Lessons Learned

- Prestimulus CCD-ITI segments contain steady-state information that can support behavioral prediction.
- A consistent preprocessing and verification workflow improves reproducibility of results.
- Integrated workflows reduce experimentation overhead

Challenges

- **Model not completed in time:** The learning procedure still fails to achieve stable performance for correctness classification.
- **Majority-class collapse:** The correctness classifier tends to predict a single class, yielding an accuracy close to chance level (approximately 50%) while providing limited practical value.
- **Class imbalance:** Correct/incorrect labels are imbalanced in parts of the data, which can bias training and evaluation without careful handling.
- **Data integrity issues:** Evidence of data corruption and trial/label misalignment can cause feature–label shifts, making supervised learning unstable and masking true signal–behavior relationships.

Future Work

- **Fix label reliability:** add dataset-level integrity checks (trial counts, event ordering, and label alignment) and remove or repair corrupted segments.
- **Handle class imbalance:** use stratified splits, class-weighted loss, resampling, and calibration to avoid majority-class collapse.
- **Improve evaluation:** report confusion matrices and class-aware metrics (F1-score, precision, recall) alongside accuracy.
- **Modeling improvements:** explore robust feature sets and temporal models after data alignment is verified.

Chapter 6

Software Development

This chapter documents how the software was developed to support the project's research workflow. It describes the development methodology, key architectural patterns used to keep experimentation maintainable, and the technology stack used to implement preprocessing, modeling, evaluation, and the supporting interface.

6.1 Software Development Methodology

The development of the proposed EEG analysis platform follows an Agile software development methodology. Agile emphasizes iterative development, flexibility, and continuous improvement, making it well-suited for research-oriented systems where requirements may evolve over time.

In the context of this project, Agile supports incremental implementation of system features, allowing the platform to be gradually refined based on testing and usability considerations. This approach is particularly appropriate for EEG data analysis systems, where workflows often require adjustment as new insights are gained during development.

The development process followed four repeating steps:

M1. Define a measurable milestone: e.g., extract CCD ITI prestimulus windows, compute features, train a regressor, or produce evaluation plots.

M2. Implement and validate: develop the smallest working version of the milestone, then validate with quick sanity checks (shape checks, trial counts, summary statistics, and train/validation splits).

M3. Evaluate and document: run a consistent evaluation protocol and record results, assumptions, and limitations.

M4. Refine: address issues found in evaluation (data leakage risks, unstable preprocessing, model underfitting/overfitting) before moving to the next milestone.

To keep the work aligned with the project scope, the model development emphasized:

- **Clear data boundaries** between subjects/sessions when creating splits.
- **Simple models first** (feature-based regressors) before attempting more complex architectures.
- **Reproducibility** through deterministic preprocessing parameters, fixed random seeds, and consistent evaluation scripts.

6.2 Technology Stack

The platform is implemented using a combination of modern web technologies and scientific computing libraries, chosen to support both user interaction and EEG data processing.

- **Frontend:** Developed using React with TypeScript, providing a responsive and structured user interface for interacting with the system.
- **Backend:** Implemented using FastAPI in Python, which handles API requests, data processing, and communication between system components.
- **EEG data processing:** Utilizes the MNE library, which provides tools for handling EEG data in standardized formats such as BIDS.
- **Artifact removal:** Employs the ASRpy library to support artifact removal through the Artifact Subspace Reconstruction method.
- **Data manipulation and computation:** Performed using NumPy and Pandas.
- **Data visualization:** Generates visual representations using Matplotlib.

- **Version control:** Git for change tracking and branching.

6.3 Coding Standards

The codebase follows a small set of conventions to keep experiments easy to reproduce and results easy to interpret.

- **Modular structure:** preprocessing, feature extraction, training, and evaluation are separated into distinct modules to reduce coupling.
- **Configuration over hard-coding:** key parameters (window length, frequency bands, feature sets, split strategy, model hyperparameters) are stored in configuration objects/files.
- **Reproducible runs:** fixed random seeds, explicit train/validation/test split definitions, and logging of dataset filters and preprocessing parameters.
- **Consistent naming:** functions and variables are named after domain concepts (trial, epoch, prestimulus window, RT target, accuracy target) rather than implementation details.
- **Input validation:** early checks for missing channels, unexpected sampling rates, and mismatched trial labels to avoid silent failures.
- **Documentation:** concise docstrings for public functions, and short experiment notes describing what changed between runs.

For evaluation outputs, plots and metrics are produced by scripts that can be rerun end-to-end, ensuring that reported results can be regenerated from the same inputs.

6.4 Progress Tracking Report

Work was tracked by incremental milestones, with each milestone producing a tangible artifact (code, plots, tables, or document sections). Table 6.1 summarizes the main development stages.

Phase	Activities	Outputs
1: Setup & scoping	Repository setup, project scope definition (CCD ITI prestimulus), and report structure alignment	Buildable LaTeX document, chapter plan
2: Data understanding	Explore HBN-EEG metadata/task structure; identify CCD ITI segments and behavioral targets (RT, accuracy)	Dataset summary notes, initial sanity plots
3: Preprocessing pipeline	Implement prestimulus window extraction and consistent preprocessing parameters; verify trial counts and split boundaries	Reusable preprocessing scripts, cached intermediate outputs
4: Model development	Train models for RT (regression) and correctness (classification); compare simple model families and feature sets	Evaluation metrics (MAE/MSE for RT; accuracy/F1/precision/recall for correctness) and prediction vs. ground-truth plots
5: Evaluation & reporting	Define evaluation protocol, error analysis, and limitations; integrate key results into deliverables chapter	Evaluation tables/figures, updated Chapters 6–7
6: UI documentation	Document the notebook-style interface used for exploration and experiment logging	UI screenshots and usage description

Table 6.1: Progress tracking summary for the prestimulus behavioral prediction project.

Chapter 7

Deliverable

This chapter summarizes the tangible outputs produced by the project and the test/evaluation protocol used to assess them. It focuses on what is delivered (software and artifacts) and how performance is measured for RT regression and correctness classification.

7.1 Software Solution

- **Project repository:** Contains preprocessing scripts and modeling code. [Link: to be added]
- **Reproducible pipeline artifacts:** dataset construction routines for CCD-ITI windows, preprocessing configurations, feature extraction scripts, and run logging.
- **Figures:** verification plots (e.g., SNR spectra) and evaluation plots used in the report; architecture diagram(s) documenting the workflow.
- **Interface documentation/prototype:** UI screenshots and descriptions supporting the experiment workflow.

7.2 Test Report

- **CCD-ITI approach:** Train on CCD-ITI segments and evaluate under a defined split strategy (preferably subject-independent).
- **Regression metrics (RT):** MAE and MSE.
- **Classification metrics (correctness):** accuracy, F1-score, precision, and recall.

- **Verification:** Use SNR spectra (and related plots) to confirm steady-state characteristics before interpreting model outcomes.

Chapter 8

Conclusion and Discussion

This chapter concludes the report by reflecting on the project outcomes, summarizing key challenges encountered during development and evaluation, and outlining directions for future improvement.

8.1 Reflection

This project set out to answer a constrained research question—whether prestimulus EEG alone can predict behavioral performance—and to deliver software that makes this question testable in a reproducible way. The main contribution is an end-to-end workflow that links (1) event-locked dataset construction for CCD-ITI windows, (2) standardized preprocessing and feature extraction for steady-state characteristics, (3) dual-target modeling (RT regression and correctness classification), and (4) consistent evaluation and reporting artifacts.

From a software engineering perspective, the outcome is a clearer separation between data ingestion, processing, modeling, and reporting. This reduces the risk that results depend on hidden notebook state and makes it easier to rerun experiments, compare configurations, and regenerate figures/tables for the final report.

8.2 Challenges

The main challenges encountered are:

- **Trial and label alignment:** prestimulus prediction depends on strict alignment between events, ITI windows, and behavioral labels; small mistakes can invalidate comparisons.

- **Data quality and non-stationarity:** EEG recordings vary across subjects and sessions, and artifacts/noise can dominate unless preprocessing and sanity checks are systematic.
- **Class imbalance and instability for correctness:** correctness labels may be imbalanced or noisy, which can lead to majority-class collapse and weak generalization.
- **Reproducibility overhead:** making experiments repeatable requires extra engineering (configs, caching, logging, deterministic splits) beyond what is needed for a one-off notebook result.

8.3 Future Work

Future work focuses on improving reliability, evaluation strength, and usability:

- **Data integrity checks:** add automated validation for event ordering, trial counts, and label sanity to detect corruption early.
- **Better handling of imbalance:** adopt stratified splitting, class weighting, calibration, and more informative analyses (e.g., confusion matrices).
- **Expanded feature/model exploration:** evaluate additional feature families and more robust models after alignment is verified.
- **Experiment tracking:** persist configurations and results in a structured format to support systematic comparison across runs.
- **Generalization to other tasks:** extend the pipeline beyond CCD if similar prestimulus windows and labels can be constructed reliably.

Reference

Bibliography

[1] B. Aristimunha, D. Truong, P. Guetschel, S. Y. Shirazi, I. Guyon, A. R. Franco, M. P. Milham, A. Dotan, S. Makeig, A. Gramfort, J.-R. King, M.-C. Corsi, P. A. Valdés-Sosa, A. Majumdar, A. Evans, T. J. Sejnowski, O. Shriki, S. Chevallier, and A. Delorme, "Eeg foundation challenge: From cross-task to cross-subject eeg decoding," NeurIPS 2025 Competition Proposal (preprint), 2025.

[2] S. Y. Shirazi, A. Franco, M. S. Hoffmann, N. B. Esper, D. Truong, A. Delorme, M. P. Milham, and S. Makeig, "Hbn-eeg: The fair implementation of the healthy brain network (hbn) electroencephalography dataset," bioRxiv, 2024.

[3] L. M. Alexander, J. Escalera, L. Ai, C. Andreotti, K. Febre, A. Mangone, N. Vega-Potler, N. Langer, A. Alexander, M. Kovacs, S. Litke, B. O'Hagan, J. Andersen, B. Bronstein, A. Bui, M. Bushey, H. Butler, V. Castagna, N. Camacho, E. Chan, D. Citera, J. Clucas, S. Cohen, S. Dufek, M. Eaves, B. Fradera, J. Gardner, N. Grant-Villegas, G. Green, C. Gregory, E. Hart, S. Harris, M. Horton, D. Kahn, K. Kabotyanski, B. Karmel, S. P. Kelly, K. Kleinman, B. Koo, E. Kramer, E. Lennon, C. Lord, G. Mantello, A. Margolis, K. R. Merikangas, J. Milham, G. Minniti, R. Neuhaus, A. Levine, Y. Osman, L. C. Parra, K. R. Pugh, A. Racanello, A. Restrepo, T. Saltzman, B. Septimus, R. Tobe, R. Waltz, A. Williams, A. Yeo, F. X. Castellanos, A. Klein, T. Paus, B. L. Leventhal, R. C. Craddock, H. S. Koplewicz, and M. P. Milham, "An open resource for transdiagnostic research in pediatric mental health and learning disorders," *Scientific Data*, vol. 4, no. 170181, 2017.

[4] N. Langer, E. J. Ho, L. M. Alexander, H. Y. Xu, R. K. Jozanovic, S. Henin, A. Petroni, S. Cohen, E. T. Marcelle, L. C. Parra, M. P. Milham, and S. P. Kelly, "A resource for assessing information processing

in the developing brain using eeg and eye tracking," *Scientific Data*, vol. 4, no. 170040, 2017.

[5] Overleaf, "Learn latex in 30 minutes," https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes.

Appendix A

Appendix A: Example

<TIP: Put additional or supplementary information/data/figures in appendices. />

Appendix B

Appendix B: About LaTeX

LaTeX (stylized as LaTeX) is a software system for typesetting documents. LaTeX markup describes the content and layout of the document, as opposed to the formatted text found in WYSIWYG word processors like Google Docs, LibreOffice Writer, and Microsoft Word. The writer uses markup tagging conventions to define the general structure of a document, to stylize text throughout a document (such as bold and italics), and to add citations and cross-references.

LaTeX is widely used in academia for the communication and publication of scientific documents and technical note-taking in many fields, owing partially to its support for complex mathematical notation. It also has a prominent role in the preparation and publication of books and articles that contain complex multilingual materials, such as Arabic and Greek.

Overleaf has also provided a 30-minute guide on how you can get started on using LaTeX. [5]